



Paradigma Orientado a Objetos

**Trabajo Práctico Cuatrimestral - Clínica Odontológica -
Informe - Entrega Final**

Profesor:

PEREYRA Jorge Agustin

Integrante:

Ferreira, Martin - 1187544

Fecha de Entrega:

23/06/2026

Curso / Comisión:

Martes - Turno tarde

Año: 2026

1. Introducción a la Interfaz Gráfica.....	3
1.1. Componentes.....	3
1.1.1. Contenedores y Controles.....	3
1.1.2. Layouts.....	3
1.1.3. Modelo de Tabla.....	3
1.1.4. Eventos.....	3
2. Manejo de Eventos.....	4
2.1. Decisiones de diseño:.....	4
3. Flujo Completo de una Operación.....	4
4. Validaciones.....	5
5. Conclusión.....	5

1. Introducción a la Interfaz Gráfica

La aplicación es una ventana principal con botones, donde cada botón abre la ventana de su sección. **VentanaPrincipal** cumple además dos roles:

Punto de composición: arma la cadena de dependencias repositorios → servicios → controladores.

Ciclo de vida: carga los datos desde archivo al iniciar y los guarda al cerrar.

- VentanaPacientes — ABM de pacientes (con domicilio).
- VentanaOdontologos — ABM de odontólogos.
- VentanaTurnos — asignación de turnos (selección de paciente y odontólogo por JComboBox), reprogramación, cambio de estado y baja.
- VentanaBusquedas — búsqueda en vivo de pacientes/odontólogos.

Cada ventana sigue la misma plantilla: formulario (arriba) + tabla (centro) + botones (abajo), lo que da un código modular y reutilizable

1.1. Componentes

Consta de varios elementos y manejo de eventos de la clase Swing de Java

1.1.1. Contenedores y Controles

JFrame, JPanel, JTable, JButton, JTextField, JLabel, JComboBox, JScrollPane, JOptionPane

1.1.2. Layouts

BorderLayout (estructura de cada ventana), GridBagLayout (formularios), FlowLayout (botones), GridLayout (menú principal)

1.1.3. Modelo de Tabla

DefaultTableModel

1.1.4. Eventos

ActionListener (botones), MouseListener (clic en fila de la tabla → carga el formulario), KeyListener (búsqueda en vivo en VentanaBusquedas), WindowListener (guardado automático al cerrar).

2. Manejo de Eventos

Cada entidad se guarda en un archivo de texto, con un registro por línea y campos separados por “;”.

- `pacientes.txt` → `id;nombre;apellido;dni;email;fechaIngreso;calle;numero;localidad;provincia`
- `odontologos.txt` → `id;nombre;apellido;matricula`
- `turnos.txt` → `id;pacienteld;odontologold;fecha;hora;estado`

2.1. Decisiones de diseño:

Modelo cargar-al-abrir / guardar-al-cerrar. La aplicación trabaja en memoria mientras está abierta; carga los archivos al iniciar y los vuelca al cerrar (vía `WindowListener`).

Los turnos guardan los IDs de paciente y odontólogo (no el objeto completo), para no duplicar datos. Al cargar, esos IDs se resuelven a los objetos reales consultando los otros repositorios. Por eso el orden de carga importa: primero pacientes y odontólogos, luego turnos.

Restauración del contador de IDs. Al cargar, cada repositorio deja su contador en el ID más alto leído, para que los nuevos registros no pisen a los existentes.

La persistencia es responsabilidad de la capa Repositorios, coherente con su rol.

3. Flujo Completo de una Operación

Ejemplo "asignar un turno": el usuario elige paciente y odontólogo en los combos, escribe fecha y hora, y presiona Asignar → la Vista captura el evento y llama al Controlador → el Controlador llama al Servicio → el Servicio valida las reglas de negocio (fecha no pasada, odontólogo libre, etc.) y, si todo está bien, crea el Turno y se lo pasa al Repositorio, que lo guarda en memoria. Al cerrar la aplicación, el repositorio vuelca todo a `turnos.txt`. Si una regla falla, el servicio lanza una excepción que la Vista atrapa y muestra con un `JOptionPane`.

4. Validaciones

Validación visual: los campos obligatorios vacíos se marcan con borde rojo antes de enviar.

Reglas de negocio: las valida el Servicio (DNI con formato correcto, email con @, unicidad de DNI/matricula, fecha del turno no anterior a hoy, etc.). Si algo no cumple, lanza la excepción correspondiente y la GUI la muestra como mensaje de error

Formato de fecha/hora: se captura `DateTimeParseException` para avisar al usuario si el formato es inválido.

5. Conclusión

El sistema cumple con los objetivos que se plantearon para la entrega, incluyendo la persistencia del proyecto mediante archivos .txt. Es un proyecto con un dominio y reglas de negocio dividido en capas para ser ordenado. También, completado en esta última etapa con una interfaz gráfica más amigable con el usuario